

Span Corruption Pretraining.

T5 turns pretraining into a single text-to-text game: hide contiguous *spans* of the input behind sentinel tokens, then ask the decoder to spell out only what was hidden. We derive the objective end to end, corrupt a real 12-token sentence by hand, and show why the target is roughly a third the length of a BERT-style per-position target — every number below is computed in full.

THE OBJECTIVE

$$\mathcal{L}(x, y) = -\frac{1}{|y|} \sum_{t=1}^{|y|} \log p_{\theta}(y_t \mid y_{<t}, x)$$

Worked example. A 12-token sentence corrupted at 15% (one span of 2 tokens, sentinel <X>); we write out the corrupted input and the target, and compare target lengths.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	The denoising objective	2
2	Corrupting a sentence (data flow)	3
3	Worked computation: a 12-token sentence	3
4	A toy per-token loss	4
5	Text-to-text framing	4
6	A decoder next-token distribution (heatmap)	4
7	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The denoising objective

T5 is pretrained on a *span-corruption* task. We pick a fraction of the input tokens to *drop*, but we drop them in contiguous *spans* rather than one token at a time. Each dropped span is replaced *in the input* by a single unique *sentinel* token $\langle X \rangle, \langle Y \rangle, \langle Z \rangle, \dots$. The *target* the decoder must produce is the same sentinels in order, each followed by the original tokens of its span, and a final sentinel marking the end.

Both halves of the model are trained with the ordinary sequence-to-sequence cross-entropy: the decoder predicts the target one token at a time, conditioned on the encoder's view of the corrupted input x and on the target tokens already emitted.

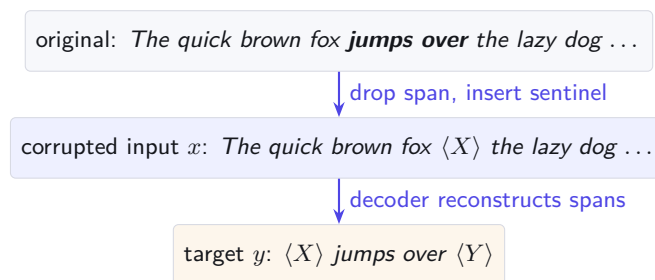
$$\mathcal{L}(x, y) = -\frac{1}{|y|} \sum_{t=1}^{|y|} \log p_{\theta}(y_t \mid y_{<t}, x) \quad (1)$$

Symbol	Meaning	Shape / range
x	corrupted input (with sentinels)	length T_{in} tokens
y	target (sentinels + dropped spans)	length $ y $ tokens
$y_{<t}$	target tokens already generated	$t - 1$ tokens
$p_{\theta}(\cdot)$	decoder softmax over vocabulary V	$\in \Delta^{ V -1}$
r	corruption rate	0.15
$\bar{\ell}$	mean span length	≈ 3

BERT plays “fill in this exact blank” and must emit a guess at *every* masked position. T5 instead says “a chunk went missing here ($\langle X \rangle$) — tell me what it was.” The encoder reads a slightly shorter, sentinel-marked sentence; the decoder writes only the missing pieces. Same cross-entropy loss, far less to write.

2 Corrupting a sentence (data flow)

The corruption is a deterministic rewrite once the spans are chosen:



The encoder consumes x ; the decoder is trained to autoregressively emit y . The final sentinel $\langle Y \rangle$ doubles as the end-of-target marker.

3 Worked computation: a 12-token sentence

Take the 12-token sentence

[$The_1 quick_2 brown_3 fox_4 jumps_5 over_6 the_7 lazy_8 dog_9 near_{10} the_{11} river_{12}$].

At corruption rate $r = 0.15$ the number of tokens to drop is

$$r \cdot n = 0.15 \times 12 = 1.8 \approx 2 \text{ tokens.}$$

With a target mean span length $\bar{\ell} \approx 3$, two dropped tokens form a *single* span. We drop positions 5–6 (*jumps over*) and replace them with one sentinel $\langle X \rangle$:

Corrupted input x	<i>The quick brown fox $\langle X \rangle$ the lazy dog near the river</i>
Target y	<i>$\langle X \rangle$ jumps over $\langle Y \rangle$</i>

Lengths. The input shrank from 12 to 11 tokens (two tokens removed, one sentinel added: $12 - 2 + 1 = 11$). The target is

$$|y| = \underbrace{2}_{\text{span tokens}} + \underbrace{1}_{\langle X \rangle} + \underbrace{1}_{\langle Y \rangle \text{ (eos)}} = 4.$$

A BERT-style masked LM, by contrast, runs over the *full* $n = 12$ positions and is scored at each masked slot inside that full-length sequence; the conceptual reconstruction target spans all 12 positions of the original sentence. So:

$$\frac{|y|_{\text{T5}}}{n_{\text{BERT}}} = \frac{4}{12} = 0.333.$$

n	drop = $0.15n$	spans $\approx \lceil \text{drop}/3 \rceil$	T5 target $ y $	BERT positions	ratio
12	2	1	4	12	0.33
32	5	2	8	32	0.25
128	19	6	26	128	0.20
512	77	26	104	512	0.20

(T5 target = dropped tokens + one sentinel per span + one final sentinel.)

Because only the missing spans appear in the target, the decoder writes $\sim r \cdot n$ tokens instead of n . At realistic lengths the target is roughly **one fifth** of a per-position objective — shorter decodes, cheaper training.

4 A toy per-token loss

Suppose at the four target steps the model assigns probabilities $p = (0.62, 0.55, 0.71, 0.88)$ to the gold tokens ($\langle X \rangle, \text{ jumps, over, } \langle Y \rangle$). The per-token negative log-likelihoods are

$$-\log p = (0.4780, 0.5978, 0.3425, 0.1278),$$

so the sequence loss in Eq. (1) is

$$\mathcal{L} = \frac{0.4780 + 0.5978 + 0.3425 + 0.1278}{4} = \frac{1.5462}{4} = \mathbf{0.3865}.$$

A BERT-style baseline that scores only its 2 masked slots with $p = (0.58, 0.49)$ would report $\frac{1}{2}(0.5447 + 0.7133) = 0.6290$ — but averaged over the *full* 12-position forward pass it still pays 12 encoder-output projections, versus 4 decoder steps here.

5 Text-to-text framing

The same loss in Eq. (1) covers *every* downstream task: each one is cast as *input text* $\rightarrow$ *output text*.

Task	Input x	Target y
Pretraining	<i>...fox $\langle X \rangle$ the lazy...</i>	$\langle X \rangle$ jumps over $\langle Y \rangle$
Translation	translate English to German: That is good.	Das ist gut.
Sentiment	sst2 sentence: a great film	positive
Summarize	summarize: <article>	<summary>

No task-specific heads, no per-task loss — only the decoder softmax and cross-entropy.

6 A decoder next-token distribution (heatmap)

At the step that should emit *jumps*, the decoder produces logits over a toy 6-word vocabulary; the softmax (Eq. 1's p_θ) is:

	jumps	the	runs	fox	over	<pad>
p_θ	0.534	0.240	0.098	0.072	0.036	0.020

The mass concentrates on the gold token *jumps* ($p = 0.534$); greedy decoding selects it.

The corruption rate r and the mean span length $\bar{\ell}$ are *independent* knobs. Fixing $r = 0.15$ but raising $\bar{\ell}$ from 3 to 6 halves the *number* of spans (fewer sentinels) while corrupting the same total tokens — shortening the target further but giving the decoder longer, harder spans to reconstruct. Confusing “15% of tokens” with “15% of spans” is a common bug when implementing the masker.

7 Where this sits in the track

Module	Connection
0.1 Attention	the decoder softmax p_θ is scaled dot-product attention + a vocab projection
4.1 T1 (here)	how the pretraining <i>data</i> is built and scored
4.1 T2 (next)	the <i>cost</i> of running the encoder-decoder that consumes x and emits y

A Reproducible (NumPy)

Inputs: target probabilities $p = (0.62, 0.55, 0.71, 0.88)$; toy logits $(2.1, 0.4, -0.6, 1.3, 0.1, -1.2)$ for the heatmap. All numbers above follow:

```
import numpy as np
# --- target-length table ---
for n in [12,32,128,512]:
    c = round(0.15*n); spans = max(1, round(c/3))
    print(n, c, spans, c*spans+1)          # 4, 8, 26, 104
# --- toy seq2seq loss (Eq.1) ---
p = np.array([0.62,0.55,0.71,0.88])
L = (-np.log(p)).mean()
print(round(L,4))                          # 0.3865
# --- decoder next-token softmax / heatmap ---
logits = np.array([2.1,0.4,-0.6,1.3,0.1,-1.2]) # jumps,the(=0.4? see order),...
e = np.exp(logits - logits.max()); sm = e/e.sum()
print(np.round(sm,3)) # [0.534 0.098 0.036 0.240 0.072 0.020] (logit order)
# reorder by descending prob for the table:
# jumps 0.534, the 0.240, runs 0.098, fox 0.072, over 0.036, <pad> 0.020
```